Fuzzing Guiado por Contrato para APIs REST

Title: Contract-Guided Fuzzing for REST APIs

Levi da Rosa Gomes (aluno)[1], Fábio Austrian Zanin (orientador)[1]

[1] Universidade Regional Integrada do Alto Uruguai e das Missões
Departamento de Engenharias e Ciência da Computação
Caixa Postal 743 – 99.709-910 – Erechim – RS – Brasil

103495@uricer.edu.br, faz@uricer.edu.br

Abstract. *This work presents a contract-guided fuzzing approach for REST APIs.*
Keywords *Word 1, Word 2, Word 3, Word 4, Word 5.*

Resumo. *Este trabalho apresenta uma abordagem de fuzzing guiado por contrato para APIs REST.*
Palavras-Chave *Palavra 1, Palavra 2, Palavra 3, Palavra 4, Palavra 5.*

1. Introdução

APIs REST tornaram-se parte essencial da infraestrutura de sistemas modernos, conectando serviços, aplicações web e dispositivos móveis [Arcuri et al. 2026]. Nesse cenário, falhas de robustez e inconsistências entre o comportamento da API e o contrato publicado impactam diretamente a confiabilidade de integrações e o custo de manutenção. Embora testes funcionais tradicionais sejam eficazes para validar cenários felizes (happy path), eles frequentemente deixam de exercitar valores-limite, formatos inválidos e combinações pouco usuais de parâmetros, que são justamente as situações em que falhas sutis tendem a se manifestar.

O uso de contratos formais, como a especificação OpenAPI (versões 3.0 e 3.1), viabiliza uma abordagem "guiada por contrato"para a geração de casos de teste: em vez de produzir entradas aleatórias, o processo explora estruturas, tipos e restrições descritas no schema para produzir mutações *válidas e inválidas* de forma sistemática. Essa estratégia aumenta a probabilidade de revelar respostas inesperadas (por exemplo, códigos 5xx, mensagens de erro inconsistentes ou divergências estruturais entre a resposta e o contrato) sem exigir acesso ao código-fonte nem instrumentação interna da aplicação, isto é, em um contexto *black-box* [Meyer 1992].

Este trabalho propõe o desenvolvimento e a avaliação de uma ferramenta de *fuzzing* automatizado para APIs REST baseada exclusivamente em contratos OpenAPI. A ferramenta implementa: (i) um parser capaz de extrair e interpretar parâmetros, corpos de requisição e restrições declaradas no contrato; (ii) um motor de mutações que gera sistematicamente casos de teste dirigidos por tipos e *constraints* (por exemplo, tamanhos mínimos e máximos, enumerações, formatação e obrigatoriedade); (iii) um orquestrador e um executor de requisições com suporte à concorrência; e (iv) um analisador que registra evidências e classifica comportamentos anômalos à luz do contrato. Além disso, o estudo inclui uma comparação exploratória entre o *fuzzing* guiado pelo contrato e uma linha de base não guiada, com o objetivo de discutir ganhos e limitações práticos da abordagem.

O escopo delimita-se a APIs REST descritas em OpenAPI, não abrangendo cenários dependentes de sequenciamento complexo de requisições ou manutenção de estado entre chamadas, autenticação avançada (por exemplo, fluxos completos de OAuth2) nem avaliação de carga ou desempenho. Em contrapartida, busca-se integrá-la de forma simples a fluxos de integração contínua (CI), reduzindo o custo de adoção e favorecendo a detecção antecipada de inconsistências durante o ciclo de desenvolvimento.

2. Justificativa

A justificativa para este trabalho reside na escassez de confiabilidade dos contratos em arquiteturas distribuídas. Embora a especificação OpenAPI seja um padrão altamente utilizado para documentação, podem existir divergências entre o que é declarado no contrato e o que a implementação de fato suporta, fenômeno conhecido como Contract Drift. Testes manuais e unitários tendem a validar o comportamento esperado, mas raramente exploram a negação do contrato, como entradas que deveriam ser rejeitadas, mas que, por falhas de sanitização ou lógica, causam exceções não tratadas (5xx).

A relevância técnica deste projeto concentra-se na automação do Teste Negativo Sistemático. Ao contrário de ferramentas de fuzzing genéricas que operam por força-bruta, este trabalho propõe que a inteligência do teste resida na semântica do schema. Ao transformar restrições estáticas (como minLength, pattern ou enum) em estratégias de mutação dinâmica, é possível identificar falhas de robustez que passariam despercebidas em fluxos de QA convencionais.

Portanto, o apelo do presente trabalho é transformar o contrato OpenAPI de um documento passivo em um oráculo ativo de testes, permitindo que equipes de desenvolvimento garantam a conformidade rigorosa de suas APIs sem o custo proibitivo de escrever milhares de casos de teste negativos manualmente.

3. Objetivos

3.1. Objetivo Geral

Desenvolver e avaliar um motor de fuzzing sintático-semântico que utilize a estrutura formal do OpenAPI para automatizar a descoberta de inconsistências de projeto e falhas de robustez em APIs REST, operando de forma estritamente black-box

3.2. Objetivos Específicos

Para dar profundidade técnica ao desenvolvimento do projeto, os objetivos específicos são:

- Implementar um tradutor de schemas para modelos de mutação: Desenvolver a lógica capaz de decompor objetos complexos e hierárquicos do OpenAPI em tipos primitivos para mapear as restrições que servirão de base para a geração de casos de teste.
- Construir um motor de mutações orientado a tipos: Criar um catálogo de heurísticas que gere payloads de borda, incluindo injeção de caracteres especiais, violações de tipos primitivos, extrapolação de limites numéricos e corrupção de estruturas JSON/YAML baseadas no schema.
- Orquestrar execuções assíncronas com controle de estado: Desenvolver um executor capaz de gerenciar requisições concorrentes e garantir a rastreabilidade entre o payload mutado e a resposta obtida, permitindo a reprodução exata das falhas encontradas.
- Automatizar a detecção de inconsistências de contrato: Implementar um módulo analisador que atue como um oráculo de conformidade, classificando automaticamente se uma resposta 4xx ou 5xx é um comportamento esperado de validação ou uma falha de implementação decorrente da falta de proteção contra entradas malformadas.
- Validar a eficácia da abordagem via análise diferencial: Comparar quantitativamente a capacidade de cobertura de erros do método guiado por contrato em relação a métodos de geração puramente aleatória, medindo a taxa de detecção de comportamentos divergentes em APIs de teste e em cenários reais.

4. Referencial Teórico

Apresente conceitos e base teórica relevante (placeholder).

5. Trabalhos Relacionados

Resuma pesquisas e soluções correlatas (placeholder).

6. Metodologia

A presente pesquisa caracteriza-se como uma pesquisa aplicada e experimental, com abordagem quantitativa para a avaliação de resultados. O trabalho foca no desenvolvimento de uma ferramenta de *fuzzing* de caixa-preta (*black-box*) orientada pela semântica de contratos OpenAPI.

6.1. Arquitetura da Solução

A ferramenta será construída de forma modular em Python, composta pelos seguintes componentes principais baseados na estrutura do projeto:

- **Parser (`openapi_parser.py`):** Responsável por ler arquivos YAML/JSON e extrair rotas, métodos e as restrições (*constraints*) de dados.
- **Motor de Mutações (`engine.py`):** Implementa a lógica de geração de *payloads*, aplicando mutações sistemáticas em tipos primitivos e estruturas complexas.
- **Orquestrador e Executor (`runner.py`):** Gerencia o plano de execução e realiza as chamadas assíncronas ao servidor-alvo.
- **Analisador (`analyzer.py`):** Atua como o oráculo do teste, comparando a resposta da API com as definições do contrato.

6.2. Etapas de Desenvolvimento

O estudo será conduzido seguindo as sete etapas detalhadas abaixo:

1. **Revisão Teórica:** Estudo aprofundado sobre a especificação OpenAPI, fundamentos de *fuzzing* e análise de ferramentas do estado da arte como Schemathesis e Dredd.
2. **Modelagem do Contrato:** Desenvolvimento da lógica de extração de metadados do OpenAPI, convertendo definições estáticas em modelos de dados para teste.
3. **Implementação dos Mutadores:** Criação de um catálogo de heurísticas para mutação de strings, números, enums e objetos, focando em violações de limites e tipos.
4. **Desenvolvimento do Executor:** Implementação da engine de requisições com suporte a concorrência e tratamento de erros de rede.
5. **Módulo de Análise:** Desenvolvimento de heurísticas para identificar inconsistências (erros 5xx) e divergências estruturais entre a implementação e o contrato.
6. **Avaliação Experimental:** Execução de testes em APIs de controle e cenários reais para medir a taxa de detecção de falhas e cobertura de *endpoints*.
7. **Discussão e Conclusão:** Análise crítica dos resultados obtidos, validação da hipótese de pesquisa e documentação de limitações da ferramenta.

7. Planejamento e Cronograma

O cronograma de atividades está distribuído entre os meses de maio e dezembro, com as etapas particionadas por semanas, conforme as normas institucionais.

Etapas / Semanas	**MAI**				**JUN**				**JUL**				**AGO**				**SET**				**OUT**				**NOV**				**DEZ**			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
1. Revisão Bibliográfica	X	X	X	X	X	X																										
2. Desenvolvimento do Parser			X	X	X	X	X	X																								
3. Motor de Mutações							X	X	X	X	X	X																				
4. Executor e Runner											X	X	X	X	X	X																
5. Módulo Analisador															X	X	X	X	X	X												
6. Testes e Experimentos																			X	X	X	X	X	X								
7. Redação do Artigo/TCC		X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X						
8. Entrega do Artigo																									X							
9. Defesa do Trabalho																										X						
10. Ajustes e Entrega Final																											X	X				

8. Resultados Esperados e Entregáveis

Liste resultados e entregas esperados (placeholder).

9. Considerações Finais

Síntese do projeto e próximos passos (placeholder).

10. References

Referências

Arcuri, A., Poth, A., Rrjolli, O., Garrett, P., e Galeotti, J. P. (2026). Fuzzing rest apis in industry: Necessary features and open problems.

Meyer, B. (1992). Applying "design by contract". *IEEE Computer*, 25(10):40–51.